

第 2 章

Cloudflare Workers と Hono

2.1 HonoではじめるCloudflare Workers

第1章では、Cloudflare WorkersおよびCloudflare Pagesの基礎について紹介しました。これまで紹介したのはあくまで基礎であり、複雑な要件に対応したアプリケーションを作成するには十分ではありませんでした。

2.1は、実践的なWebアプリケーションをCloudflare Workersで実装するための選択肢として第一候補に挙がるHonoについて紹介します。

Honoとは

Hono^{注1}はWebアプリケーションフレームワークの1つで、あらゆるJavaScriptランタイム上で動作することを特徴としています。もちろんCloudflare Workers上で動作しますし、Fastly ComputeやAWS Lambdaなどのサーバーレス環境や、Node.js、Deno、Bunでも動作します。yusukebeさん^{注2}が2021年末ごろから開発を行っており、最近ではGitHub上で10,000スターを超えるほどの人気を誇るプロジェクトとなっています。

最もシンプルなHonoアプリケーションの例

まずは、Honoの最もシンプルなアプリケーションの例を見てみましょう。**リスト1**は、Honoの公式ドキュメントの冒頭^{注3}で紹介されている例です。

リスト1 最もシンプルなHonoアプリケーション

```
import { Hono } from 'hono'
const app = new Hono()

app.get('/', (c) => c.text('Hono!'))
```

注1 <https://hono.dev/>

注2 <https://github.com/yusukebe>

注3 <https://hono.dev/top>

```
export default app
```

インポートしたHonoクラスをapp変数として初期化し、必要なルーティングの設定を行って、最後にdefault exportしています。なんと、たったこれだけのコードで、URLにアクセスするとHono!という文字列が返ってくるWebアプリケーションとなり、そのままCloudflare Workersにデプロイすることができます。

Honoの特徴

あらゆるJavaScriptランタイム上で動作する

Honoは、冒頭でも紹介したとおり、あらゆるJavaScriptランタイム上で動作します。

従来、JavaScriptでWebサーバの開発を行う場合は、Node.js上でFastifyやExpressなどを使用するのが一般的でした。近年は、Node.jsだけでなく、BunやDeno、Cloudflare WorkersといったさまざまなJavaScriptランタイムが登場しています。Bunはリリース当初からNode.jsへの高い互換性を謳っており、DenoもNode.jsの互換機能を強化しています^{注4}が、完全に互換性があるとは言いきれません。そのため、Node.js向けに作られたライブラリやフレームワークが、すべてのJavaScriptランタイム上で動作するとは限りません。

一方で、各JavaScriptランタイム間で共通したAPIも存在します。ECMAScriptの仕様で定義されているAPIと、Web API^{注5}です。Web APIは、本来ブラウザ上で動作するJavaScriptのために策定されていたものですが、Denoが積極的にWeb APIの実装を進めたのを皮切りに、Node.jsでも実装が進み、あらゆるJavaScriptランタイム間で共通して利用可能なAPIとみなされるようになっていきました。2022年には、WinterCG^{注6}が発足し、JavaScriptランタイム間で共通APIとして実装すべきWeb APIのサブセットを標準化し、

注4 <https://deno.com/blog/v1.15#improving-node-compatibility>

注5 https://developer.mozilla.org/ja/docs/Learn/JavaScript/Client-side_web_APIs/Introduction

注6 <https://blog.cloudflare.com/introducing-the-wintercg-jp>

相互運用可能なプログラムを記述できるようにするための動きが生まれています。

Honoは、特定のランタイムに依存した機能を使わず、Fetch APIなどのWeb APIをベースに開発が行われているため、あらゆるランタイムで動作することが可能となっています。ただし、一部APIに互換性のないランタイムに関しては、別途アダプターが提供されています。アダプターが必要になる場合も、コードの差分は最小限となっています。**リスト2**は、公式ドキュメントに記載されているNode.js アダプターの使用例^{注7}です。

リスト2 Node.js アダプターを使用したHonoアプリケーションの例

```
import { serve } from '@hono/node-server'
import { Hono } from 'hono'

const app = new Hono()
app.get('/', (c) => c.text('Hello Node.js!'))

serve(app)
```

高速

Honoは速度を重視して開発されています。とくに、ルーティングには強いこだわりがあり、あらゆるルート設定や、ランタイムの特性に適した方法を使い分けられるように、複数のルーターの実装を持っています。多くの環境で最も速いのはRegExpRouterという正規表現ベースのルーターですが、ルートの登録は高速ではないため、リクエストのたびにルートが登録されるようなランタイムではLinearRouterが適しているとのことです。ルーティングの詳細^{注8}については公式ドキュメントを参照してください。

公式のベンチマーク^{注9}においては、Honoの使用するルーターがCloudflare WorkersおよびDeno上で動作するものの中で最も高速であることが示されて

注7 <https://hono.dev/getting-started/nodejs>

注8 <https://hono.dev/concepts/routers>

注9 <https://hono.dev/concepts/benchmarks>

います。Honoは、Deno公式のベンチマーク^{注10}にも採用されています。

軽量

Honoは、Web APIベースで作られており、外部のライブラリへの依存がないため非常に軽量です。特にhono/tinyプリセットを利用すると14KB未満となります。比較対象として示されているExpressは572KBとなっています^{注11}。

Cloudflare WorkersをはじめとするCDNエッジ上のランタイムでの利用を考えたとき、ワーカーのファイルサイズはコールドスタートにかかる時間に影響することがあるため重要です。また、ランタイムによっては、ワーカーのファイルサイズに制限がある場合もあります。ファイルサイズに制限があるケースでは、Webアプリケーションの中核となるフレームワークのサイズが小さいほど、ほかのパッケージを選択する幅が広がるため、Honoは魅力的な選択肢となります。

充実したミドルウェア

Honoは、ExpressやFastifyなどと同様に、ミドルウェアの機構を持っています。ミドルウェアの実装、および設定はシンプルな方法でできます。**リスト3**は、Honoの公式ドキュメントで示されているミドルウェアの設定例^{注12}です。ここでは、ETagとLoggerの設定を行うミドルウェアがすべてのルートに対して設定されています。

リスト3 Honoのミドルウェアの設定例

```
import { Hono } from 'hono'
import { etag } from 'hono/etag'
import { logger } from 'hono/logger'

const app = new Hono()
app.use('*', etag(), logger())
```

注10 <https://deno.com/benchmarks>

注11 <https://hono.dev/top#lightweight>

注12 <https://hono.dev/top#middleware-helpers>

Honoには、ビルトインのミドルウェアと、別パッケージとして分けて提供されるミドルウェア（公式ドキュメント上では、Third-party Middlewareと呼ばれています）が数多く存在します。ビルトインのミドルウェアには、ベーシック認証、圧縮（gzip/deflate）、キャッシュなどがあります。別パッケージとして提供されるミドルウェアには、Firebase Authenticationや、Zodバリデーターなどがあります。Honoでアプリケーションを実装する際は、自分の作りたい機能を実現するミドルウェアがすでに存在していないかを公式ドキュメント上で確認するとよいでしょう。

TypeScript連携

HonoはTypeScriptの型定義が充実しています。とくに、URLパラメータに対する型チェックおよび補完は強力です。たとえば、**リスト4**のようにルーティングを行った場合、リクエストパラメータを取得するparamメソッドの呼び出し時に、利用可能なパラメータ名がString Literal Typeとして推論されます。

リスト4 URLパラメータ取得メソッドの型推論の例

```
app.get("/users/:id", async (c) => {  
  c.req.param("id") // "id"のString Literal Typeとして推論される  
})
```

Honoの内蔵するHTTPクライアントを使用すると、Zodによって定義されたスキーマをもとに、Honoサーバとクライアントで型定義を共有することもできます。公式ドキュメントでは、Honoサーバの使用する型定義をもとに、HonoのHTTPクライアントとReact Query^{注13}を組み合わせるリクエストを行う例^{注14}が示されています。

Cloudflare Workersとの自然な連携

HonoはあらゆるJavaScriptランタイム上で動作しますが、必要に応じて、各ランタイムで最適な開発体験を実現するための機能提供が行われています。

Cloudflare Workersでの開発体験も自然なものとなっています。たとえば、

注13 <https://tanstack.com/query/latest/docs/react>

注14 <https://hono.dev/concepts/stacks>

環境変数、R2/KVなどへのバインディングについて見てみましょう。Honoを使わない場合、Cloudflare Workersに渡された環境変数の取得は**リスト5**のように実装します。

リスト5 Cloudflare Workersの環境変数取得の実装

```
type Env = {  
  MESSAGE: string  
}  
  
export default {  
  async fetch(req: Request, env: Env): Promise<Response> {  
    return new Response(`API host: ${env.MESSAGE}`)  
  },  
}
```

ここでは、Fetch Handlerに渡ってきたenvオブジェクトに設定されているMESSAGEキーから環境変数の値を取得しています。R2やKVへのバインディングも、同様にenvオブジェクトから取得します。envオブジェクトに対してのTypeScriptによる型付けは自動的に行われないため、明示的に型を宣言して指定する必要があります。

Honoを利用する場合は、**リスト6**のようにハンドラーに渡ってくるc.envオブジェクトから環境変数を取得します。

リスト6 Honoを使った環境変数取得の実装例

```
import { Hono } from "hono"  
type Bindings = {  
  MESSAGE: string  
}  
  
const app = new Hono<{ Bindings: Bindings }>()  
app.get("/*", (c) => {  
  return c.text(`API host: ${c.env.MESSAGE}`)  
})  
export default app
```

環境変数の取得および、c.envオブジェクトへの型の指定について、素のCloudflare Workersのものと似通った実装になっていることがご理解いただけたと思います。この体験が、Cloudflare Workers向けに特別な設定をすること

なく得られる点は、Honoの魅力の1つと言えるでしょう。

HonoとCloudflare WorkersでHello, World!

続いて、実際にHonoのプロジェクトを作成して、Cloudflare Workersにデプロイしてみましょう。このガイドは、Honoの公式ドキュメントのGetting Started^{注15}をもとに執筆しています。

事前準備

必要なものは次のとおりです。

- Cloudflare アカウント
- Node.js
- npm

執筆にあたって、Node.jsは最新のLTS (20.11.0) を、npmはバージョン10.3.0を使用しました。

プロジェクトの作成

Honoのプロジェクトの作成は非常に簡単です。ターミナル上で、プロジェクトの作成に使用したいディレクトリに移動し、次のコマンドを実行してください。

```
$ npm create hono@latest hello-hono
```

すると、Which template do you want to use?と尋ねられますので、ここではcloudflare-workersを選択します。すると、hello-honoディレクトリが作成されるので、次のコマンドのように、作成されたディレクトリに移動して、必要なパッケージをインストールしましょう。

注15 <https://hono.dev/getting-started/basic>

```
$ cd hello-hono  
$ npm install
```

ローカル開発環境での動作確認

続いて、Honoプロジェクトのローカル開発環境での動作確認を行います。`npm run dev`コマンドを実行しましょう。すると、Wranglerによってローカルの開発サーバが立ち上がります。この状態で、bを入力すると、ブラウザ上に“Hello Hono!”と表示されます。1.2でCloudflare Workersのプロジェクトを作成する際に紹介したように、ターミナルでcurlコマンドが使える環境の方は、`curl 127.0.0.1:8787`を実行して動作確認することもできます。

デプロイ

デプロイは、`npm run deploy`コマンドで実行できます。初めてWranglerを使う方は、ここでまず自身のCloudflareアカウントに対する認証を行ってください。コマンドを実行すると、hello-hono.サブドメイン.workers.devのURLでアクセス可能な形でデプロイが行われます。ここまで完了したら、ブラウザでこのURLを開くか、curlコマンドなどでリクエストを行って、ローカル開発環境と同様に動作していることを確認しましょう。

Honoプロジェクトの構成

`npm create hono`コマンドによって作成されるHonoプロジェクトの構成は次のとおり、とてもシンプルです。

```
├── README.md  
├── package.json  
├── src  
│   └── index.ts  
├── tsconfig.json  
└── wrangler.toml
```

package.json内で、src/index.tsをdevコマンドおよびdeployコマンドで使用するよう設定されているため、アプリケーション開発時に編集するのは基

本的にsrcディレクトリ内のみとなります。TypeScriptやWranglerの設定は初めから行われています。wrangler.tomlには最低限の設定しか行われていないので、KVやR2などを利用する場合は設定内容を編集してください。



Honoの主な用途

Cloudflare Workers上でのHonoの主な用途としては次の2つが考えられます。

- プロキシサーバの実装
- Webサーバの実装



プロキシサーバの実装

Cloudflare Workersは、もともとオリジンサーバに対するHTTPリクエスト／レスポンスを加工したり、複雑なロジックを使ったキャッシュを行ったりといったプロキシサーバとしての用途が中心でした。Honoを活用することでプロキシサーバの実装が簡単になるので、Cloudflare Workersの魅力を最大限に引き出すことができます。

Honoによるプロキシサーバは、**リスト7**のように実装します。

リスト7 Honoによるプロキシサーバの実装

```
import { Hono } from 'hono'
const app = new Hono()
app.all('*', (c) => fetch(c.req.raw))
export default app
```

ここで行っているのは、Cloudflare Workersが受け取ったリクエストを、そのままFetch APIに流し、得られたレスポンスを返す処理のみです。たったのこれだけで、Cloudflareを前段に挟んだオリジンサーバへのリクエスト／レスポンスが、すべてこのHonoアプリケーションを経由する形を実現できます。

続いて、ここにベーシック認証を追加する例を考えてみましょう。Honoにはベーシック認証のミドルウェアがあるので、先ほどのプロキシサーバに**リスト8**のコードを追加して、/auth配下のパスにアクセスがあった場合はベーシッ

ク認証をかけることができます。

リスト8 Honoによるベーシック認証プロキシの例

```
import { basicAuth } from 'hono/basic-auth'
(..略..)
app.use(
  '/auth/*',
  basicAuth({
    username: 'user',
    password: 'password',
  })
)
```

このように、任意のWebサイトの全体のパスや、特定のパスのパターンに対して共通の処理を行いたいようなときに簡単に設定できます。

事前に作られたミドルウェアを活用するのはもちろん、自分でプロキシの実装を行いたいときも、Cloudflare Workers単体で実装するより、高機能なルーターが組み込まれているHonoを利用したほうが便利なケースが多いでしょう^{注16}。



Webサーバの実装

Honoは、ExpressやFastifyと似た用途として、Webサーバの実装でも利用することができます。

ルーティングの設定は、app.getなどのメソッド呼び出しで行います(**リスト9**)。対応するHTTPメソッドを指定しない場合は、app.allメソッドの呼び出しでルートを追加します。

リスト9 ルーティングの設定例

```
app.get('/', (c) => c.text('GET'))
app.post('/', (c) => c.text('POST'))
app.put('/', (c) => c.text('PUT'))
app.delete('/', (c) => c.text('DELETE'))
app.all('/', (c) => c.text('ALL'))
```

注16 yusukebeさんの執筆したプロキシパターンの紹介記事で、Cloudflare Workers上でHonoによるプロキシサーバを実装する例が多数示されているので、興味のある方はぜひこちらをご覧ください。
<https://zenn.dev/yusukebe/articles/647aa9ba8c1550>

ルートには、`/users/:id`のようにURLパラメータを含むことができますし、`/posts/:date{[0-9]+}`のように正規表現を利用することも可能です。パスのグルーピングを行いたい場合は、Honoインスタンスをネストすることで実現できます (**リスト10**)。

リスト10 ルーティングをグルーピングする例

```
const users = new Hono()
users.get('/:id', (c) => { ... })
users.post('/', (c) => { ... })

const app = new Hono()
app.route('/users', users)
```

レスポンスは、ハンドラー関数に渡るContextオブジェクトのメソッドを呼んで返します。ヘッダーの設定には`c.header`メソッドを、HTTPステータスコードの設定には`c.status`メソッドを、レスポンスボディの設定には`c.body`メソッドを使用します。**リスト11**は、公式ドキュメント^{注17}に記載されているハンドラーの設定例です。

リスト11 レスポンスの設定例

```
app.get('/welcome', (c) => {
  c.header('X-Message', 'Hello!')
  c.header('Content-Type', 'text/plain')
  c.status(201)
  return c.body('Thank you for coming')
})
```

テキスト、JSON、HTMLをレスポンスボディとして返す場合は、`c.text`、`c.json`、`c.html`メソッドがそれぞれ利用可能です。これらを使用すると、必要なContent-Typeヘッダーを自動で設定してくれます (**リスト12**)。

注17 <https://hono.dev/api/context#body>

リスト12 JSONレスポンスを返却する例

```
app.get('/', (c) => {
  return c.json({ message: 'Hello, World!' })
})
```

JSON APIサーバとしての用途では基本的に`c.json`を利用すればよいのですが、これだけでは表現力に限界があるので、より実践的なAPIサーバを実装したい場合は、Zod OpenAPIやGraphQLサーバのミドルウェアが用意されているので^{注18}、これらの使用を検討するとよいでしょう。

またHonoは、HTMLの記述のために、JSX記法を提供しています^{注19}。HTMLをテンプレートとして扱い、一部分を差し替えてレスポンスを返す動的なWebサイトを構築したい場合は、**リスト13**のように`c.html`メソッドと併せてJSXを使用するのがよいでしょう。JSXの利用には`tsconfig.json`への追加設定が必要ですが、`npm create hono`コマンドによって生成されるHonoプロジェクトでは初めからこの設定が行われているため不要となります。

リスト13 JSXによって組み立てたHTMLを返却する例

```
const Page = (
  <html>
    <body>Hello, World!</body>
  </html>
)
app.get('/', (c) => c.html(<Page />))
```

注18 Zod OpenAPI : <https://github.com/honojs/middleware/tree/main/packages/zod-openapi>
GraphQLサーバ : <https://github.com/honojs/middleware/tree/main/packages/graphql-server>

注19 <https://hono.dev/guides/jsx>



2.1 まとめ

2.1 は、Hono の特徴、プロジェクト作成方法および、主な用途について紹介しました。Hono は、Cloudflare Workers の活用の幅をぐっと広げてくれる強力なツールです。ここで紹介したのは基本的な機能でしたが、Hono の開発は非常に活発で、より高度な機能が次々と追加されています。たとえば、最近 JSX 内に CSS を記述するためのヘルパー関数なども追加されました^{注20}。あらゆるシーンに活用できる、非常に柔軟な Web フレームワークですので、ぜひこの内容を参考にご自身で試してみてください。

注 20 <https://hono.dev/helpers/css>



2.2 Hono で作る短文投稿 Web アプリ

2.1 は、Hono の初歩について解説しました。2.2 は、Hono と Cloudflare スタックを組み合わせる Web サーバの開発例について紹介します。Web サーバの開発については 2.1 でも簡単に紹介しましたが、ここでは Cloudflare の提供する D1 というリレーショナルデータベースを使用した、より具体的な例について説明します。



Hono + Cloudflare スタックでの Web サーバ開発のメリット

実装方法の詳細に踏み込む前に、まずは、Hono + Cloudflare スタックで Web サーバを開発するメリットについて過去の記事の内容をふまえておさらいしましょう。

1.1 で紹介した「Cloudflare Workers の特徴」には、次のようなものがありました。

- 低コスト
- グローバルな環境へのデプロイ

まず、コストについてですが、Workers の有料プラン（月々 5 ドル）には 1 日あたり 1,000 万リクエスト分の費用が含まれており、それを超える分は 100 万リクエストごとに \$0.3 の課金となっています。Hono を利用する場合はこの低コスト基盤の恩恵を受けることができますが、Express.js などを実装された Node.js アプリケーションサーバを実行する場合、これだけ安価な基盤を探すことは難しいでしょう。

次に、グローバルな環境へのデプロイについてです。これによって、デプロイしたアプリケーションがすぐに世界中のユーザーから近い距離のエッジサーバで実行されるようになりますし、リクエスト数に応じて最適な数の Worker

が起動するため、スケールも柔軟です。Webサーバの実行に専用のインスタンスを用意するケースでは、リクエストを受け付ける間サーバが稼働し続ける必要があります。また、リクエスト数の増大による負荷上昇に備えて、スケールアップのしくみを整えておく必要があります。Cloudflareスタックで開発を行えば、Webサーバが呼び出すキーバリューストア (KV) などグローバルな環境で自動的にスケールしてくれます^{注1}。

Honoは、これらのCloudflare Workersのメリットを活用したWebサーバの開発を行うのに最適なWebアプリケーションフレームワークとなっています。

HonoによるWebサーバ開発方法の選択肢

2.1でも簡単に紹介しましたが、HonoによるWebサーバ（とくに、今回はバックエンドのAPIサーバの開発について取り上げます）の開発には、いくつかの選択肢が存在します。

- 素のHonoによる実装
- Zod Validatorミドルウェアの利用
- Zod OpenAPI Honoミドルウェアの利用
- GraphQL Serverミドルウェアの利用
- tRPC Serverミドルウェアの利用

これらの中で最も手軽なのは、素のHonoによる実装を除くと「Zod Validatorミドルウェアの利用」です。スキーマバリデーションライブラリであるZodによる定義を使い、API呼び出しに対するバリデーションの実行と、ハンドラー実装内での型チェックを行います。このパターンは、バリデーションの実行という実利と、開発体験の良さのバランスがよくとれているのでお勧めです。ここでは、こちらの方法での実装を紹介します。

^{注1} ただし、Cloudflareの提供するサービスの一部は特定のリージョン下のみで動作するもの（R2など）も存在する点には注意しましょう。

ここでは紹介しませんが、Zod Validatorミドルウェアを使用したハンドラーの型定義は、フロントエンドでも使い回すことができます。Hono製のAPIサーバへのリクエストを行うフロントエンドのコードを、型定義を使って安全に実装できるということです。このパターンはHonoのドキュメントではRPC機能^{注2}として紹介されており、Hono Stacks^{注3}という別ページでも言及されている重要なものです。フロントエンドからバックエンドのハンドラーの型定義を使用する際に使うHonoのHTTPクライアント（hono/client）は、特定のフロントエンドのライブラリに依存していないので、ReactやVueなど、好きなライブラリと組み合わせて使用することができます^{注4}。

Zod OpenAPI Honoミドルウェア^{注5}も魅力的です。このミドルウェアを使用すると、Honoのハンドラー定義をもとにOpenAPIのスキーマを生成できます。OpenAPIのスキーマがあれば、JavaScript/TypeScriptに限らない、任意のプログラミング言語によるクライアントを生成することができるので、活用の幅が広がります。これに加えて、Swagger UIミドルウェア^{注6}を使用すれば、Zod OpenAPIミドルウェアによって生成されたスキーマに対応するSwagger UI^{注7}を表示することができます。以上のとおり、OpenAPIを使った開発に期待される多くの機能を持ち合わせているので、こちらも有力な選択肢となるでしょう。GraphQL Serverミドルウェアと、tRPC Serverミドルウェアについての紹介は割愛します。

短文投稿Webアプリを作ってみよう

HonoとD1を組み合わせて、短文投稿の機能を実現するWebアプリケーションを開発します。バックエンドのAPIサーバだけではなく、投稿を行うためのフォームと、投稿一覧を表示する画面も合わせてすべてHonoで実装を行

^{注2} <https://hono.dev/guides/rpc>

^{注3} <https://hono.dev/concepts/stacks>

^{注4} 公式ガイドでは、React HooksライブラリのSWRと組み合わせて使う例が紹介されています。
<https://hono.dev/guides/rpc#using-swr>

^{注5} <https://github.com/honojs/middleware/tree/main/packages/zod-openapi>

^{注6} <https://github.com/honojs/middleware/tree/main/packages/swagger-ui>

^{注7} <https://swagger.io/tools/swagger-ui/>

います^{注8}。

事前準備

必要なものは次のとおりです。

- Cloudflare アカウント
- Node.js
- npm

執筆にあたって、Node.jsは最新のLTS（20.11.1）を、npmはバージョン10.4.0を使用しました。

プロジェクトの作成

今回のプロジェクトは、単純なAPIサーバの実装だけではなく、投稿一覧ページの表示もふまえて作成する必要があります。このような、HTMLを組み立てて結果を返したいようなケースでは、HonoのCloudflare Pages用のテンプレートを使うとよいでしょう。Cloudflare Pages用のテンプレートには、初めからJSXによってページ全体のレイアウトの設定が行われているため、自分でHTMLタグ、HEADタグ、BODYタグなどの構造を組み立てる必要がありません。

ターミナル上で、プロジェクトの作成に使用したいディレクトリに移動し、次のコマンドを実行してください。

```
$ npm create hono@latest posts-app
```

すると、Which template do you want to use?と尋ねられますので、ここではcloudflare-pagesを選択します。すると、posts-appディレクトリが作成されるので、次のコマンドのように、作成されたディレクトリに移動して、必要なパッケージをインストールしましょう。

^{注8} 今回作成するプロジェクトの完成形のコードは、筆者のリポジトリから確認することもできます。
<https://github.com/syumai/sd-cloudflare-examples/tree/main/posts-app>

```
$ cd posts-app  
$ npm install
```

ハンドラーの設定の確認

具体的な実装に進む前に、プロジェクトの初期状態で設定されているハンドラーの内容を確認しましょう。src/index.tsxを開くと、リスト1のように設定されています。

リスト1 初期状態のハンドラー設定（抜粋）

```
app.get('*', renderer)  
  
app.get('/', (c) => {  
  return c.render(<h1>Hello!</h1>)  
})
```

すべてのパスに対して指定されているrendererは、HonoのJSX Rendererミドルウェア^{注9}の設定です。ここで、JSXを使ったレイアウトの描画設定を行います。/のパスに対して指定されているハンドラーの内容は、rendererミドルウェアによって設定されたレイアウトの内側に描画されます。

この設定内容をそのまま活かし、/postsのパスにアクセスしたら投稿一覧が表示される形を目指していきましょう。

D1データベースのセットアップ

今回のアプリケーションの核となる、D1データベースのセットアップを行います。手順としては、次の流れとなります。

1. テーブルスキーマの記述
2. データベースの作成
3. ローカル環境へのマイグレーションの実行
4. データベース本体へのマイグレーションの実行

^{注9} <https://hono.dev/middleware/builtin/jsx-renderer>

まず、テーブルスキーマの記述を行います。今回は、非常にシンプルな短文投稿アプリケーションを作成しますので、**リスト2**のように、id、title、bodyだけを持つpostsテーブルを作成する形で記述します。D1はSQLiteベースのデータベースなので、SQLの文法についてはSQLiteのものを参照ください。**リスト2**の内容をエディタに記述したら、posts-appディレクトリの直下にschema.sqlというファイル名で保存してください。

リスト2 postsテーブルのスキーマ定義

```
DROP TABLE IF EXISTS posts;
CREATE TABLE IF NOT EXISTS posts (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  title TEXT,
  body TEXT
);
```

データベースの作成は、Wranglerを使って行います。次のコマンドを実行してposts-appという名前のデータベースを作成してください。

```
$ npx wrangler d1 create posts-app
```

実行が完了すると、作成されたD1データベースの情報が出力されるので、これをコピーして、posts-appディレクトリの直下にwrangler.tomlとして保存してください。加えて、wrangler.tomlの末尾にpreview_database_id = "DB"という記述を追加して保存してください。ここまでの作業を行うと、wrangler.tomlの内容は**リスト3**のようになります。

リスト3 wrangler.tomlの内容

```
[[d1_databases]]
binding = "DB" # i.e. available in your Worker on env.DB
database_name = "posts-app"
database_id = "データベースID"
preview_database_id = "DB"
```

ここで注意すべき点ですが、Cloudflare Pagesの開発において、wrangler.

tomlがWranglerによって考慮されることはありません。そのため、今保存したファイルに含まれるDBというバインディングは、自動的にPages Functionのプログラムから使用可能とはなりません。バインディングの追加は手動でCloudflareのコンソール上から行う必要があるため、こちらの作業は最後に行います。wrangler.tomlを今回ここで保存したのは、D1データベースに対してのマイグレーション実行をWrangler経由で行うためです。

続けて、ローカル環境、データベース本体のそれぞれにマイグレーションを実行します。次のコマンドを実行してください。

```
$ npx wrangler d1 execute posts-app --local --file=./schema.sql
$ npx wrangler d1 execute posts-app --file=./schema.sql
```

ここまでで、D1データベースのセットアップは完了となります。

データベースを使った開発設定の追加

Cloudflare Pagesのローカル開発でD1データベースを使えるようにするためには、別途設定が必要となります。HonoのCloudflare Pagesテンプレートから作成したプロジェクトの場合は、vite.config.tsに修正を行います。公式ドキュメントを参考に、**リスト4**のように変更します^{注10}。

リスト4 修正後のvite.config.tsの内容

```
import build from '@hono/vite-cloudflare-pages'
import devServer from '@hono/vite-dev-server'
import { defineConfig } from 'vite'
import { getEnv } from '@hono/vite-dev-server/cloudflare-pages'

export default defineConfig({
  plugins: [
    build(),
    devServer({
      entry: 'src/index.tsx',
      env: getEnv({
        d1Databases: ['DB'], // ①
        d1Persist: '.wrangler/state/v3/d1', // ②
      })
    })
  ]
})
```

注10 <https://hono.dev/getting-started/cloudflare-pages#bindings>

```

    },
  },
],
})

```

d1Databases: ['DB']の箇所（**リスト4の①**）で、ローカルの開発環境のプログラム中でどのような名前でデータベースにアクセスを行うかというバインディングの設定を行っています。

d1Persist: '.wrangler/state/v3/d1'の箇所（**リスト4の②**）は、ローカル開発環境で使用するデータベースの内容がどこに永続化されるかを指定しています。ここで指定したのは、先ほどWranglerを使ってローカル環境に対して行ったマイグレーションの実行結果のデータベース内容が保存されているパスです。このパスの指定を行わないと、デフォルトの設定で別のパスを参照してしまうので注意してください^{注11}。

🔗 投稿作成ハンドラーの記述

いよいよ、投稿の作成を行うハンドラーを記述していきます。まずは、Honoインスタンス経由でD1データベースを参照できるようにするために、バインディングの型定義を追加しましょう。src/index.tsxを開いて、app変数の宣言を**リスト5**のように変更します。

リスト5 Honoインスタンスへのバインディングの追加

```
const app = new Hono<{ Bindings: { DB: D1Database } }>()
```

これで、ハンドラー関数内でc.env.DBといった形でD1データベースへ参照できるようになります。

次に、必要なpackageのインストールを行います。次のコマンドで、ZodとZod Validatorミドルウェアを追加しましょう。

注11 WranglerのgetPlatformProxy関数でより簡潔に設定ができますが、追加されて間もないため、ここでは使用しません。
<https://developers.cloudflare.com/workers/wrangler/api/#getplatformproxy>

```
$ npm install zod @hono/zod-validator
```

続けて、src/index.tsx ファイル内にimport文、createPostSchema変数の宣言、/postsハンドラーの設定を追加してください（**リスト6**）。

リスト6 投稿作成ハンドラーの実装

```
import { zValidator } from '@hono/zod-validator'
import { z } from 'zod'

(...略...)

const createPostSchema = z.object({
  title: z.string(),
  body: z.string(),
})

app.post('/posts', zValidator('form', createPostSchema), async (c) => {
  const { title, body } = c.req.valid('form')
  const sql = 'INSERT INTO posts (title, body) VALUES (?, ?)'
  await c.env.DB.prepare(sql).bind(title, body).run()
  return c.redirect('/posts')
})
```

createPostSchema変数の宣言（**リスト6の①**）は、投稿作成リクエストの内容に対するZodのスキーマ定義です。投稿は、タイトルと本文のみを含むので、title、bodyの2値をそれぞれ文字列型で受け取る想定としています。

app.post('/posts' 以下（**リスト6の②**）は、投稿作成ハンドラーの実装となります。HTTPメソッドのPOSTに対応する形で処理を登録したいので、このハンドラーはapp.postを使用しています。

②の部分を掘り下げて説明します。まず、zValidator('form', createPostSchema)の箇所で、Zod Validatorミドルウェアの使用を行うこと、リクエストをフォーム形式で受け取ること、そしてスキーマにcreatePostSchema変数の内容を使用することを指定しています。formの箇所にはリクエストボディとしてJSONを受け付けたいときに使うjsonや、URLクエリを受け付けたいときに使うqueryなども指定できます。ここで行ったミドルウェアの設定を利用して、後続のハンドラー関数の記述時にc.req.

`valid('form')`の形式で、`createPostSchema`で指定された形式のオブジェクトを取得することが可能となります。

ハンドラー関数内では、D1データベースに対するSQLの実行を行っています。SQL内では、`?`の形式でプレースホルダーを使用することができます。プレースホルダーに対するバインディングは`bind`メソッドの呼び出しで行い、バインディング結果のSQLを`run`メソッドの呼び出しで実行します。

最後に、これから作成する投稿一覧画面（GET HTTPメソッドで`/posts`への登録を想定）にリダイレクトして終了しています。

🔍 投稿一覧画面のハンドラー記述

投稿一覧画面には、一覧だけでなく、投稿フォームもセットで表示を行います。これらに必要なコンポーネントの定義を`src/index.tsx`に追加していきましょう。追加する位置は、先ほどリスト6で追加した`app.post('/posts')`ハンドラーの後ろにしてください。

まずは、投稿フォームのコンポーネントを追加します。リスト7の`PostForm`コンポーネントは単純なHTMLのフォームです。actionには先ほど設定した投稿作成ハンドラーのURLを設定し、HTTPメソッドとしてPOSTを指定します。フォームの見た目は、1行にタイトルの入力フォーム、本文の入力フォーム、投稿ボタンが並ぶ形式となります。

リスト7 PostFormコンポーネントの定義

```
const PostForm = () => (
  <form method='post' action='/posts'>
    <input name='title' placeholder='title' />
    <input name='body' placeholder='body' />
    <button>Post</button>
  </form>
)
```

続けて、投稿一覧のコンポーネントを追加します。リスト8の`PostsList`コンポーネントは、非常に簡素ですが、プロパティとして受け取った`posts`変数の内容をひとつひとつ変換してHTMLとして出力しています。併せて、コンポーネントのプロパティ定義で必要になるPost型の定義をここで行っています。

リスト8 Post型とPostsListコンポーネントの定義

```
type Post = {
  title: string
  body: string
}

const PostsList = ({ posts }: { posts: Post[] }) => (
  <div>
    {posts.map((post) => (
      <div>
        <div>Title: {post.title}</div>
        <div>Body: {post.body}</div>
        <hr />
      </div>
    ))}
  </div>
)
```

最後に、投稿一覧のハンドラー関数を追加します。リスト9のハンドラー関数は、D1データベースから`posts`テーブルの内容をすべて取得し、その結果を使って`PostsList`コンポーネントを描画します。併せて、`PostForm`コンポーネントの描画も行い、ページの表示を完了します。このように、HonoのJSX Rendererミドルウェアを使うことで、React.jsなどを使った経験のある方であれば親しみやすいJSXの形式でコンポーネント化したHTML要素を組み立てていくことが簡単にできます。

リスト9 投稿一覧のハンドラー関数

```
app.get('/posts', async (c) => {
  const posts = await c.env.DB.prepare('SELECT * FROM posts').all<Post>()
  return c.render(
    <div>
      <PostForm />
      <PostsList posts={posts.results} />
    </div>
  )
})
```

ここまでで、すべての実装は完了となります。

動作確認

これまで実装した結果がうまく動作するか、ローカル環境で確認してみましょう。次のコマンドで、ローカルの開発サーバを起動してください。

```
$ npm run dev
```

すると、Viteの開発サーバが起動するので、表示されるLocal:の項目に指定されたURLを使い、**http://localhost: ポート番号 /posts**にアクセスしてみてください。投稿用のフォームが表示され、投稿自体も問題なく行うことができれば成功です。

実際にCloudflare Pagesにデプロイして動作確認するには、次のコマンドを使用します。

```
$ npm run deploy
```

新しいプロジェクトを作成するかどうかの質問については、「Create a new project」を選択し、名前に「posts-app」を指定してください。production branch nameについては、お使いのリポジトリに合わせてmainなどを指定してください。

デプロイが完了したら、続けてD1データベースのためのバインディングの追加を行います。Cloudflareのダッシュボード^{注12}にアクセスし、「Workers & Pages」の項目から、作成されたposts-appプロジェクトを探してください。

posts-appプロジェクトの「Setting」タブから、「Functions」を選択し、その中にある「D1 database bindings」の項目から[Add binding]をクリックし、Variable nameとして「DB」を、D1 databaseとして「posts-app」を選択して[Save]をクリックすると、設定は完了です。

production branch nameに指定したbranch上でデプロイを実行すると、Production環境へのデプロイが行われ、Cloudflare PagesのURLが発行されます。URLは、Cloudflareのダッシュボードのposts-appプロジェクトのトップ

注12 <https://dash.cloudflare.com/>

ページに表示されます。このURLを使って動作確認できたら、無事、プロダクションでのD1データベースの利用に成功したことになります。

ただし、今回のアプリケーションは、バリデーションもほぼ行っていない簡素なもので、URLを共有する際にはご注意ください。

2.2 まとめ

2.2は、HonoとD1を使って、実際にCloudflareスタック上で前回よりも実践的なWebサーバの開発を行いました。Cloudflare PagesでD1などのバインディングを使うには少し癖がありますが、HonoとZod Validatorミドルウェアによるハンドラーの記述や、JSX RendererミドルウェアによるHTMLテンプレートの記述は非常にスムーズに行えるということがご理解いただけたと思います。

また、ここでは紹介していませんが、HonoのJSX関連機能にはClient Componentsもあり、フロントエンドのアプリケーション開発も可能となっています^{注13}。直近では、Honoの機能を活用したフルスタックのWebアプリケーションフレームワークのHonoXも登場しています。Hono^{注14}は、バックエンド用途に限らず、非常に活発に開発が行われているので、ぜひ開発の動向を追ってみてください。

注13 <https://hono.dev/guides/jsx-dom>

注14 <https://github.com/honojs/honox>

